Project Plan: Educational Plan-and-Solve Chatbot

(RL + RAG + Knowledge Gain)

1. Project Overview

This document outlines a complete, end-to-end project plan for creating an educational "Plan-and-Solve" chatbot. The chatbot will specialize in helping users learn programming (Python/C) by generating step-by-step reasoning plans, not just final solutions.

This project's core innovation is the use of Reinforcement Learning (RL) guided by a "knowledge gain" reward signal, integrated with a Retrieval-Augmented Generation (RAG) system to provide rich, educational context for each step.

2. Core Objectives

- **Educational Focus:** To develop a planner that optimizes for *educational reasoning* rather than just solution correctness.
- **Novel Reward:** To design and implement a "knowledge gain" reward metric that encourages the model to produce clear, conceptually-rich, and non-redundant plans.
- **Integrated System:** To build an end-to-end system where the RL-trained planner guides a RAG retriever, which in turn feeds a synthesizer (LLM) to produce a final, context-rich answer.
- **Evaluation:** To rigorously evaluate the educational effectiveness of this approach against baselines (e.g., SFT-only, RL-without-knowledge-gain).

3. Novelty & Strategic Impact

This approach is highly novel and shifts the paradigm from "just solve" to "teach how to solve."

- **New Focus:** Most code LLMs optimize for Pass@k (correctness). This plan optimizes for *reasoning* and *teaching*.
- **New Signal:** "Knowledge gain" as an RL reward signal is a new approach, moving beyond simple correctness or human preference scores.
- **New Combination:** The integration of RLHF (with a custom reward) + RAG + reasoning plans for tutoring is an unexplored research area.
- **Research Potential:** This project could lead to significant contributions in AI in Education, RL for Reasoning, and Explainable AI (XAI).

4. Feasibility & Complexity Analysis

Overall Feasibility: Doable (Moderate-to-High Complexity)

The project is feasible as a research prototype. Data is available (APPS, CodeContests), and the RL and RAG components are well-understood.
Production-level deployment would introduce further challenges, including real-time retrieval, scalable inference, and continuous learning from student feedback.

Key Challenge:
The most significant and abstract challenge is the Reward Modeling. Designing an accurate proxy for "knowledge gain" is non-trivial. The initial approach will use semantic similarity and concept coverage, with the potential to later integrate real student performance feedback.
Component Complexity Breakdown:

Component	Complexity	Rationale
Data Preparation	**High**	Extracting/generating reasoning traces and concept tags is resource-intensive.
Reward Model	**Medium-High**	Designing the "knowledge gain" metric is novel and abstract.
RL Fine-Tuning	**Medium**	The PPO pipeline is documented, but requires careful hyperparameter tuning.
RAG Integration	**Low-Medium**	Standard vector DB setup (FAISS, ChromaDB) and retrieval logic.
Evaluation	**Medium**	Requires a multi-faceted framework beyond simple correctness metrics.

5. Key Considerations & Strategic Inputs (Thoughtful Input)

Before the detailed plan, here are three high-level strategic inputs:

1. Input 1: The Reward Model Dilemma (Proxy vs. Correctness)
   The "knowledge gain" metric (Phase 2) is an intrinsic proxy for educational value. However, a plan step could be "novel" but functionally wrong. This could lead the RL agent to learn to generate creative but incorrect plans.

    - **Mitigation:** This plan *must* combine the intrinsic r_knowledge with an *extrinsic* reward for correctness. The simplest is a sparse reward (e.g., +1.0) if the *entire* plan, when synthesized, leads to a solution that passes unit tests. The final reward in Phase 5 should be a weighted sum: R_final = w1 * r_knowledge + w2 * r_correctness.
2. Input 2: The Planner's "Scope of Knowledge"
   The SFT model (Phase 3) is trained on plans, not solutions. This is correct. It means the RL agent (Phase 5) is learning how to reason, not how to code. This is a crucial distinction. The small model (Phi-3-mini) is perfect for this, as it's fast and we are only asking it to generate ~5-7 steps, not 100 lines of code. The RAG system (Phase 6) then provides the "knowledge" (the code) that the planner doesn't have. This separation of concerns is the plan's greatest strength.
3. Input 3: The Synthesizer's "Faithfulness"
   The final step (Phase 6) relies on a large LLM to synthesize the answer. A major risk is that this LLM will ignore the provided plan and retrieved context, and just solve the problem from its own pre-trained knowledge. This would invalidate the entire pipeline.
    - **Mitigation:** Prompt engineering for the synthesizer is critical. The prompt must *force* the model to be "faithful" to the inputs. See the added input in Phase 6 for a specific prompt template.

6. Proposed System Architecture

The end-to-end system will follow this logical flow:

1. **User Query:** User asks a programming problem (e.g., "How do I merge two sorted lists?").
2. **Planner (RL Model):** The planner_rl_model receives the query and generates a step-by-step reasoning plan.
    - Plan:$$1. Init pointers, 2. Create result list, 3. Compare & append, ...$$
3. **Retriever (RAG):** For each step in the plan, the retriever queries the vector DB (using problem_text + step_text) to find relevant educational content (tutorials, code snippets).
4. **Synthesizer (LLM):** A large LLM (e.g., Gemini, GPT-4) receives the original problem, the generated plan, and all retrieved context.
5. **Final Answer:** The Synthesizer generates a complete, educational answer that explains the solution using the plan and the retrieved context.

7. Detailed Project Plan (Phased Breakdown)

🧱 PHASE 1: Data Preparation

- **Goal:** Prepare reasoning-style programming data with plan steps and solutions.
- **Actions:**
    1. **Obtain Raw Dataset:** Source from codeparrot/apps. Optionally supplement with MBPP, HumanEval, or CodeContests.
       from datasets import load_dataset
       # APPS is a good source for reasoning-style problems
       ds = load_dataset("codeparrot/apps")

2. **Extract Fields:** For each problem, store problem_text, canonical solution, input_output test cases, and difficulty in a JSONL file.
   ```
   {
    "problem_text": "Write a function to merge two sorted lists.",
    "solution": "def merge_sorted(l1, l2):\n  res = []\n  i = j = 0\n  ...",
    "tests": [{"input": "[1,3],[2,4]", "output": "[1,2,3,4]"}],
    "difficulty": "easy"
   }
   ```

3. **Generate Stepwise Plans:** Use an LLM (as a tutor persona) to generate step-by-step plans for each problem.
   - **Prompt Template:**
     You are a master programming tutor helping a student plan their solution.
     DO NOT write the code. Just explain the step-by-step plan a student should follow.

     Problem:
     <problem_text>

     Step-by-step plan:

   - **Store as:** "plan_steps": ["1. Understand input/output", "2. Initialize two pointers (i, j) to 0", ...]
4. **Generate Concept Tags:** Use an LLM or regex to tag key programming concepts.
   - **Store as:** "concept_tags": ["two-pointer", "list", "merge", "iteration"]
5. **Build Knowledge Base (for RAG):** Scrape or collect open-source programming tutorials (e.g., "Python By Example", GeeksforGeeks conceptual pages).
   - Split into small, meaningful chunks (~300 words).
   - Create embeddings (e.g., text-embedding-3-small or a sentence-transformers model).
   - Store in a vector DB (FAISS, ChromaDB).
6. **Create Unified Dataset:** Combine all fields into a final JSONL.

🧠 PHASE 2: Knowledge Gain Metric Setup

- **Goal:** Create a function that measures the conceptual knowledge contributed by each plan step.
- **Actions:**
  1. **Define "Conceptual Distance":** Use embeddings to compute the novelty of a step relative to the previous step.knowledge_gain = cosine_sim(step, solution) - cosine_sim(step, previous_step)
     - A positive score means the step introduced new semantic information relevant to the solution.

2. **Define "Coverage Reward":** Reward a step if it introduces a new concept from concept_tags not covered by previous steps.coverage_gain = len(new_concepts_in_step) / total_concepts
    - This rewards explicit coverage of the problem's key topics.
3. **Combine & Normalize:** Create a final per-step *intrinsic* reward:r_knowledge = α * knowledge_gain + β * coverage_gain (e.g., α=0.7, β=0.3)

🤖 PHASE 3: Supervised Fine-Tuning (SFT) of the Planner

- **Goal:** Teach a small LLM (e.g., Qwen-1.8B, Phi-3-mini) to produce reasoning plans.
- **Actions:**
    1. **Format Training Data:** Create input/output pairs from the dataset.
        - **Input:**
          Problem: Write a Python function to merge two sorted lists.

        - **Output:**
          1. Initialize indices i, j = 0 for both lists.
          2. Create an empty list `res` to store the merged result.
          3. Loop while `i` is in bounds of list1 AND `j` is in bounds of list2.
          4. Inside the loop, compare elements at `list1[i]` and `list2[j]`.
          5. Append the smaller element to `res` and increment its corresponding index (i or j).
          6. After the loop, append any remaining elements from list1.
          7. Append any remaining elements from list2.
          8. Return the merged list `res`.

    2. **Fine-Tuning:** Use Hugging Face trl (SFTTrainer). Train for 3-5 epochs with a low learning rate (1e-5).
- **Output:** planner_sft_model

🦾 PHASE 4: Reward Model (RM) Training

- **Goal:** Train a small model that predicts the "knowledge gain" reward for a given step.
- **Actions:**
    1. **Create Reward Dataset:** For each (problem, step, solution), compute the r_knowledge (from Phase 2). Store as (input_text, reward_value).
       {
         "input": "Problem: merge two lists\nPlan so far: 1. Init pointers\nStep: 2. Create empty result list\nSolution: ...",
         "reward": 0.8
       }

    2. **Train RM:** Use a small transformer (e.g., distilbert-base) with a regression head. Minimize Mean Squared Error (MSE) between predicted reward and computed r_knowledge.

- **Output:** reward_model

🏋️ PHASE 5: RL Fine-Tuning (PPO / DPO)

- **Goal:** Use RL to teach the SFT planner to maximize high-knowledge-gain plans.
- **Actions:**
  1. **Initialize PPO Agent:** Load planner_sft_model as the base policy and reward_model to provide the *intrinsic* reward signal.
  2. **Run Episodes:**
     - Sample a problem.
     - Generate plan steps sequentially.
     - For each step, compute r_knowledge using the reward_model.
     - **(Thoughtful Input)** At the end of the *full plan*, generate a solution (using the Phase 6 pipeline) and run unit tests.
     - Calculate a final *extrinsic* reward: r_correctness = 1.0 if tests pass, 0.0 otherwise.
     - Combined Reward: The total reward for the episode is a combination: R_total = (sum of all r_knowledge for each step) + w * r_correctness (e.g., w=10.0 to strongly reward correctness).
     - Update the planner policy with PPO.
  3. **Validation:** Periodically evaluate on a held-out set.
- **Output:** planner_rl_model

🔍 PHASE 6: RAG Integration & Synthesis

- **Goal:** Use the planner to guide retrieval and generate the final answer.
- **Actions:**
  1. **Generate Plan:** Pass the user query to the planner_rl_model.
  2. **Retrieve:** For each plan step, query the vector DB (query = problem_text + step_text) to get top-K documents.
  3. **Generate Answer:** Pass all components to a large synthesizer LLM.
  4. **(Thoughtful Input) Synthesizer Prompting:** Use a prompt that *forces faithfulness* to the plan.
     You are a helpful programming tutor.
     A student has asked for help with the following problem:
     <problem_text>

     You MUST follow this exact step-by-step plan:
     <plan_steps>

     To help you, here is relevant documentation for each step:
     <retrieved_context>

     Now, generate a complete, educational answer. For each step in the plan, explain the step clearly and provide the corresponding code, using the retrieved documentation to help you.

🧩 PHASE 7: Evaluation Framework

- **Goal:** Measure the success of the system based on correctness, plan quality, and educational value.
- **Metrics:**
  1. **Core Performance:** Pass@k (proportion of problems solved correctly).
  2. **Plan Efficiency:** Average number of plan steps (shorter, concise plans are better).
  3. **Intrinsic Plan Quality:**
     - **Completeness:** Does the plan cover all necessary steps?
     - **Logical Order:** Are steps in the correct sequence?
     - **Non-redundancy:** Are there circular or repeated steps?
  4. **Educational Value & Alignment:**
     - **Knowledge Gain Mean:** Average r_knowledge score per plan.
     - **Clarity:** Can a learner understand the reasoning? (Human evaluation).
     - **Semantic Similarity:** Compare embeddings of plan steps vs. final solution.
- **Baselines for Comparison:**
  1. SFT-only planner (no RL).
  2. RL planner *without* knowledge gain (e.g., optimizing for r_correctness only).
  3. The final planner_rl_model (with r_knowledge + r_correctness).

💬 PHASE 8: Chatbot Integration & Deployment

- **Goal:** Deploy the full pipeline into the educational assistant.
- **Actions:**
  1. **Implement API Pipeline:** Create a single API endpoint that orchestrates the full flow.

```
def answer_question(user_query):
    # Phase 5: Planner generates the plan
    plan = planner_rl_model.generate_plan(user_query)

    # Phase 6: Retrieve context for each step
    all_context = []
    for step in plan:
        query = user_query + " " + step
        context = retriever.query(query, top_k=3)
        all_context.append(context)

    # Phase 6: Synthesizer generates the final answer
    final_answer = synthesizer_llm.generate_faithful_answer(
        problem=user_query,
        plan=plan,
        context=all_context
    )
    return {"plan": plan, "answer": final_answer}
```

2. **(Thoughtful Input) UX Improvement:** Instead of returning one giant answer, the chatbot should be *interactive*.
    - Show Step 1 of the plan.
    - Show the explanation and code for Step 1.
    - Ask the user: "Does that make sense? Shall we move to the next step?"
    - This makes the experience truly "tutor-like" and respects the user's learning pace.
3. **Feedback Loop (Optional):** Add "Was this helpful?" buttons. This user feedback can be collected and used as a *real-world* reward signal (replacing the proxy r_knowledge) to fine-tune the reward_model and planner_rl_model over time.

8. Summary Roadmap

Phase	Goal	Output
1	Prepare Data (APPS + reasoning + tags)	Structured dataset & Vector DB
2	Build Knowledge Gain Function	Reward computation pipeline
3	Supervised Train Planner	planner_sft_model
4	Train Reward Model	reward_model
5	RL Fine-Tuning (PPO/DPO)	planner_rl_model
6	Integrate RAG Retrieval	Retriever + Synthesizer pipeline
7	Evaluate and Analyze	Metrics & improvement report
8	Deploy in Chatbot	End-to-end system API